fwd user guide

Siddharth M. Bhatia
August 8, 2026

Contents

fwd user guide 3
Getting started 4
- What fwd does 4
- Install 4
- Launch a session 4
- Configure a target 5
- Everyday workflow 6

Commands and lifecycle 7
- Launch and attach 7
- Durable tasks 7
- Inspect sessions 8
- Synchronize and compare 8
- Continuous synchronization 9
- Forward local ports 10
- Manage configured targets 10
- Stop, remove, and uninstall 11

Configuration and backends 12
- Configuration layers 12
- Default command 12
- Target setup 12
  - Manage saved targets 13
- SSH 13
- RunPod 14
- Lambda Cloud 14
- Slurm 15
- Project setup and synchronization 15

Coding agents 17
Launch Claude Code or Codex remotely 17
Send follow-up turns 17
Install the fwd skill 18
Runtime policy 18
Troubleshooting 19
Start with diagnostics 19
Recover a partial launch 19
Stopped compute 19
Dirty remote worktree 20
Synchronization limits 20
Destructive operations 20

fwd user guide

This folder contains the user documentation for `fwd`. Start with the guide that matches what you want to do:

- Getting started: install fwd, create a target, and launch your first remote session.
- Commands and lifecycle: launch, attach, run durable tasks, synchronize files, forward ports, stop, and remove sessions.
- Configuration and backends: layered configuration, SSH, RunPod, Lambda Cloud, Slurm, defaults, and project setup.
- Coding agents: move Claude Code or Codex work, send follow-up turns, manage credentials, and install the fwd skill.
- Troubleshooting: diagnose failures, recover partial launches, and avoid data loss.

The installed CLI is the authoritative option reference:

```
fwd --help
man fwd
fwd COMMAND --help
fwd config --example
fwd config --schema
```

The first invocation of an installed `fwd` silently synchronizes its bundled manuals to the user-local XDG man directory; package upgrades update them on the next use. Run `man fwd` or `man fwd-up` afterward. In a source checkout, preview a page directly with `man ./man/fwd.1`.

Contributing to fwd? See the developer documentation.

Getting started

What fwd does

`fwd` moves a local coding project to remote compute. It provisions or connects to a target, synchronizes the project, prepares detected Python, JavaScript, or Swift tooling, and starts a persistent shell, command, Claude Code, or Codex session in tmux.

The remote session survives a dropped SSH connection or closed laptop. You can reconnect later, run durable background tasks, and pull results back to the local checkout.

Install

```
uv tool install fwdit
```

On the first `fwd` invocation, the bundled `fwd(1)` and command manuals are silently installed under the user-local XDG man directory, normally `~/.local/share/man/man1`. Later package upgrades update changed pages and remove obsolete fwd pages automatically, without `sudo` or a global post-install hook:

```
fwd --version
man fwd
man fwd-up
```

To try fwd without installing it:

```
uvx --from fwdit fwd --help
```

Local requirements are Python 3.12+, `ssh`, and `rsync`. Provider targets may require their own authentication or CLI; `fwd doctor` reports what is missing.

Launch a session

From a project directory:

```
fwd                         # attach to this project's session, or create
     one interactively
fwd up runpod codex         # launch Codex on a RunPod target without
     attaching
fwd up --target work        # launch the configured default command on
     target "work"
fwd up -- python train.py   # launch and stream a durable command
```

Bare `fwd` is the human-friendly reuse workflow. It attaches to an unambiguous session or creates one interactively. Scripts and coding agents should use explicit, non-attaching `fwd up` forms.

Detach from tmux with `Ctrl-B D`; the remote process continues running. Reconnect with:

```
fwd ls
fwd attach SESSION
```

Configure a target

You can use a direct SSH host, an OpenSSH alias, or built-in CPU RunPod defaults without saving a target:

```
fwd up --target user@example.com
fwd up --target my-ssh-alias
fwd up runpod
```

For reusable or provider-specific settings, run:

```
fwd targets add
```

`fwd targets add` (also spelled `fwd setup`) only writes configuration; it does not provision compute. Review saved targets with `fwd targets ls`, and edit one in place with `fwd targets update NAME`. See Configuration and backends for non-interactive setup and provider requirements.

Everyday workflow

```
fwd ls                    # inspect this project's sessions
fwd send -- pytest -q     # run durable work in the current remote
     session
fwd diff                  # compare synchronized local and remote files
fwd pull outputs/         # retrieve selected results without deleting
     local files
fwd stop                  # stop the session after the remote worktree
     is clean
fwd rm                    # permanently destroy its remote resources
```

Continue with Commands and lifecycle, or read Coding agents when moving a Claude Code or Codex workflow.

Commands and lifecycle

Launch and attach

```
fwd                              # human reuse workflow: attach or
     create interactively
fwd up                           # launch the configured default;
     stay local
fwd up runpod codex              # positional target and coding agent
fwd up --target work --agent codex
fwd up --new codex               # create a second session for
     this project
fwd up -- python train.py --epochs 10
fwd attach SESSION               # human terminal only
```

`fwd up` is idempotent and also repairs partially prepared sessions. Use `--new` only when you want a separate remote resource. Agent launches attach automatically only in a human terminal; `--detach` always stays local.

An explicit command runs as a durable task and returns its exit status. After two seconds, `Ctrl-C` cancels that task and `Ctrl-B` backgrounds the local viewer. `fwd attach` enters the primary tmux session; detach with `Ctrl-B D`.

Durable tasks

`fwd send` runs work in an existing session and never starts or restarts compute:

```
fwd send -- pytest -q
fwd send --name SESSION --detach -- python train.py
fwd send --ls --json
fwd send TASK_ID                 # replay the log and continue following
fwd send TASK_ID --stop          # cancel only this task
fwd send cancel                  # cancel all queued tasks
fwd send cancel all              # cancel all active tasks
```

Use -- before commands whose flags belong to the remote program. Invoke a shell explicitly for pipes, redirects, and globs:

```
fwd send -- bash -lc 'cat outputs/*.json | jq .'
```

On supported backends, `--stop-after` queues remote-owned shutdown after new work completes. `fwd send stopafter` queues shutdown after all current work, and `fwd send cancel stopafter` disarms it. Lambda does not support remote stop-after; use local `fwd stop` after retrieving results.

Inspect sessions

```
fwd ls
fwd ls --all-projects
fwd ls --json
fwd ls --columns backend,status,ports
fwd doctor --json
fwd info --json
```

Rich tables are used in terminals. Redirected or agent output defaults to Markdown; `--json` provides stable machine-readable output. Progress and diagnostics stay on stderr.

Commands accept exact session names and, where unambiguous, target or backend aliases. Use selectors positionally with `attach`, `stop`, `rm`, and `diff`; use `--name` with `send`, `push`, and `pull`. `stop` and `rm` accept multiple selectors.

Synchronize and compare

```
fwd diff                          # unified diff; no changes to either side
fwd diff -q                       # exit 0 same, 1 different, 2 error
fwd push                          # mirror local files to remote
fwd pull                          # additive whole-project download
fwd pull outputs/ logs/           # additive path-scoped download
```

Synchronization honors `.gitignore`, `.fwdignore`, and configured exclusions. Upload includes `.git/` so remote agents retain repository context; pull and diff exclude `.git/`, so pulling does not transfer remote commits. Export a patch or Git bundle when you need a remote commit locally.

Push mirrors the local synchronized tree and normally deletes remote-only synchronized files. Pull is always additive and never deletes local files. Uploads are capped by `sync.max_size_gb` (1 GB by default) and use a staging

directory so an interrupted or over-limit upload does not replace the live remote project.

Continuous synchronization

`fwd sync` keeps the local project and the remote directory converged in both directions while a session runs, instead of waiting for an explicit `push` or `pull`. It is built on Mutagen, is **off by default**, and is stored per target:

```
fwd sync on                      # enable for this target and start it now
fwd sync status                  # configured intent, live state,
     and conflicts
fwd sync status --json
fwd sync off                     # stop syncing; the remote directory is
     left as-is
fwd sync on SESSION              # act on a named session instead of
     this directory's
```

Toggling takes effect immediately, including on a session that is already running. Enabling it also enables it for future launches on that target; `fwd up` and `fwd attach` start the sync automatically whenever it is on.

Key properties:

- **`.git` is never continuously synced.** Both sides run Git concurrently, and a half-propagated index or packfile corrupts the repository. `fwd push` and `fwd pull` still carry `.git`, and remain the way to move repository state.
- **Conflicts are never resolved destructively.** The mode is Mutagen's `two-way-safe`, so a file edited on both sides is reported by `fwd sync status` rather than overwritten. Settle one by deleting the copy you do not want on one side; the surviving version then propagates. Editing the losing side again does not help — that is simply another competing change.
- **The initial transfer still goes through rsync.** `fwd up` performs its usual size-bounded push, and Mutagen only moves steady-state deltas afterwards, so `sync.max_size_gb` still governs the large transfer.
- **Ignore rules are captured when the sync starts**, translated from `.gitignore` (including nested files), `.fwdignore`, and `sync.exclude`.

After editing those, run `fwd sync off && fwd sync on` to pick up the change.

- **Not every transport supports it.** Mutagen installs a remote agent over `scp`, which the RunPod SSH proxy cannot run; fwd warns and skips continuous sync there.

fwd installs and drives Mutagen itself: the remote agent installs automatically on first connection, and the first local use offers to run `brew install mutagen-io/mutagen/mutagen` when Homebrew is available. fwd runs its own isolated Mutagen daemon under `~/.fwd/mutagen`, so it never disturbs a Mutagen daemon you run yourself.

Forward local ports

```
fwd ports 3000                  # local 3000 to remote 3000
fwd ports work 8080:3000        # local 8080 to remote 3000
fwd ports --ls
fwd ports --close 3000
fwd ports work --close          # close all forwards for one session
fwd up --ports 3000 --ports 8080:3000
```

Forwards bind local and remote loopback only. fwd refuses occupied local ports before changing tunnel state and tracks persistent OpenSSH control masters across CLI processes.

Manage configured targets

`fwd targets` manages saved targets — the `[targets.NAME]` configuration entries that describe *how to reach* compute. The session commands above manage the compute itself.

```
fwd targets ls                  # name, backend, key connection detail,
     default marker
fwd targets ls pod              # only targets whose name contains "pod"
fwd targets add ssh --host my-box --target-name work
fwd targets info work           # resolved values, which are defaults, and
     tracked sessions
fwd targets update work         # setup again, prefilled with the current
     values
```

```
fwd targets rm work            # remove the configuration entry only
```

`fwd setup` remains a permanent alias for `fwd targets add`. All of these read local configuration and session state only; no provider is contacted. `update` and `rm` open a picker when no target is named and fail with an actionable message when nothing can answer a prompt; `rm` requires `--force` non-interactively. Removing a target never stops or destroys compute — use `fwd rm` for that — and removing the default repoints `default_target` at the sole survivor or clears it.

Stop, remove, and uninstall

```
fwd stop SESSION               # stop compute; retain configured
     persistent storage
fwd rm SESSION                 # destroy remote resources and forget
     the session
fwd rm one two                 # operate on several resolved sessions
fwd rm --all                   # destroy every tracked session
     after confirmation
fwd uninstall                  # remove local fwd data, skill,
     and completion
```

`stop` and `rm` check the remote Git worktree and refuse when reachable work is dirty. Resolve that work with `fwd diff`, `fwd pull`, or a remote commit before retrying. Force flags explicitly accept possible data loss.

`fwd rm` is irreversible. `fwd uninstall` does not destroy remote resources and should normally follow `fwd rm --all`.

See Troubleshooting for recovery and safety details.

Configuration and backends

Configuration layers

User configuration lives at `~/.fwd/config.toml`. A project `.fwd/config.toml` deep-merges over it; target-specific settings have the highest precedence.

Use generated output instead of copying a static exhaustive example:

```
fwd config                          # effective values and their
     source files
fwd config --example                # complete commented TOML
fwd config --example runpod         # one backend
fwd config --schema                 # JSON Schema for tools and editors
fwd config set sync.delete false
fwd config set --project forwarding.ports 3000 8080:3000
fwd config rm --project forwarding.ports
```

Default command

Claude is the built-in default for a newly created session. Change it by scope:

```
fwd default codex
fwd default --project claude
fwd default --target runpod -- python -m agent
```

Precedence is target, project, user, then built-in Claude. An explicit agent or command on `fwd up` overrides the default for that launch.

Target setup

Humans can use the interactive wizard:

```
fwd targets add
fwd targets add runpod
```

`fwd setup` is a permanent alias for `fwd targets add` and accepts exactly the same arguments.

Scripts and coding agents should pass all required values. Missing values produce the exact flags needed instead of opening a prompt:

```
fwd targets add ssh --host my-box --target-name work
fwd targets add runpod --data-center-id US-GA-1 --target-name pod
fwd targets add lambda --instance-type gpu_1x_a10 --ssh-key-name my-
    public-key --target-name lambda-gpu
fwd targets add slurm --login-host login.example.edu --user me --remote-
    base /scratch/me/fwd
```

Manage saved targets

`fwd targets` operates on the `[targets.NAME]` tables in configuration; `fwd ls` and `fwd rm` operate on the sessions running against them.

```
fwd targets ls                  # name, backend, key connection detail,
    default marker
fwd targets ls pod              # only targets whose name contains "pod"
fwd targets info work           # resolved values, which are defaults,
    and tracked sessions
fwd targets update work         # re-run setup prefilled with the
    current values
fwd targets update work --host new-box   # non-interactive single-field
    edit
fwd targets rm work             # remove the config entry only
```

`update` and `rm` open a picker when no target is named, and fail with the available names when one is misspelled or when nothing can answer a prompt. `rm` requires `--force` non-interactively, mirroring `fwd config rm`. Removing the default target repoints `default_target` at the sole remaining target, or clears it when the choice would be a guess. Removal never touches remote compute or session state: destroy those with `fwd rm`.

Inspect currently valid provider machine strings with `fwd up --machines` or `fwd up TARGET --machines`. A one-launch `--machine/-m` value must exactly match an available provider value.

SSH

SSH targets connect to machines you already control. Prefer an OpenSSH host alias so `~/.ssh/config` owns the user, port, identity, and proxy settings:

```
fwd up --target my-box
fwd targets add ssh --host my-box --target-name work
```

RunPod

RunPod requires `runpodctl` 2.6.0 or newer and an authenticated account. CPU is the compute default. Persistent Secure Cloud storage is enabled by default for CPU and GPU sessions; `fwd stop` terminates the disposable Pod while retaining the per-session network volume, and `fwd rm` deletes both. `persistent = false` opts into disposable storage.

Persistent targets require a datacenter. GPU targets also require an exact GPU choice and an appropriate image:

```
fwd targets add runpod --target-name pod --data-center-id US-GA-1
fwd up pod --machines
```

Lambda Cloud

Lambda uses its HTTPS API directly and requires a locally available `LAMBDA_API_KEY`, instance type, and registered SSH key. Interactive setup can securely store a pasted key or a reference to a key file. The API key is never copied to the remote instance.

Persistent filesystems are enabled by default and survive instance termination. `fwd stop` terminates compute while retaining storage; `fwd rm` deletes both. Lambda cannot perform remote stop-after because that would require copying the broad account API key to the instance.

```
export LAMBDA_API_KEY='...'
fwd targets add lambda --target-name lambda-gpu --instance-type gpu_1x_a10
     --ssh-key-name my-public-key
fwd doctor --target lambda-gpu
```

Slurm

Slurm targets connect through a login node and run the allocation inside persistent tmux. Use shared scratch storage for the project, tools, and caches; do not use a quota-constrained home directory.

```
fwd targets add slurm --target-name hpc --login-host login.hpc.example.edu
     --user me --remote-base /scratch/me/fwd
```

Allocation, partition, account, proxy jump, and environment-module settings are cluster-specific. On Slurm, ordinary `fwd send` commands run on the login node; invoke `srun` when work belongs inside an allocation.

Project setup and synchronization

fwd detects Python, JavaScript, and Swift Package Manager projects, reuses compatible remote tools, and installs missing requirements in user-controlled persistent locations. Commit an idempotent `.fwd/setup.sh` for project-specific setup that built-in toolchains do not cover.

Common synchronization settings:

```
[sync]
exclude = [".venv", "node_modules", "dist"]
use_gitignore = true
delete = true
max_size_gb = 1.0
continuous = false          # Mutagen-backed continuous sync; per target:
     [targets.NAME] continuous_sync
[forwarding]
ports = ["3000", "8080:3000"]
```

Setting `sync.exclude` replaces the default list rather than extending it. See `fwd config --example` for every field and current default.

`sync.continuous` is the default for continuous synchronization. Any target may override it with `continuous_sync`, which is deliberately three-state: omit it to inherit `sync.continuous`, or set `true`/`false` to decide for that target regardless of the global default.

```
[sync]
continuous = true            # most targets sync continuously

[targets.hpc]
backend = "slurm"
continuous_sync = false      # ...except this one
```

`fwd sync on` and `fwd sync off` write exactly this key for you, so it rarely needs editing by hand. Continuous sync never includes `.git`; use `fwd push` and `fwd pull` for repository state.

Coding agents

Launch Claude Code or Codex remotely

```
fwd up --target runpod --agent codex
fwd up work claude
fwd up --no-attach codex
```

Claude transfers the current transcript by default so the remote CLI resumes the same conversation. `--handoff` creates a summarized `HANDOFF.md` instead. `--user-config` copies portable Claude configuration; `--creds` copies live Claude authentication and should be used only when the user explicitly accepts storing it on the remote disk.

Codex copies portable configuration, rules, and skills, but not `~/.codex/auth.json`. Authenticate on the remote when required. Supported Codex installations also start the managed Remote Control app-server beside the terminal TUI. Supported Claude accounts can expose the same interactive conversation through Claude Remote Control. Missing support or authentication does not block the tmux session.

GitHub authentication setup is enabled by default for development targets. Disable it with `--no-setup-github` or `[github] auth = false` when credentials must remain local.

Send follow-up turns

```
fwd send agent "continue the implementation"
fwd send agent --detach "run the long evaluation"
fwd send agent --stop "replace the current approach"
fwd send agent --stop-after "finish, save results, and stop compute"
```

Follow-up turns use the existing remote conversation and durable task log. Normal turns queue in order; `--stop MESSAGE` interrupts the active turn and sends a replacement. `--stop` without a message interrupts the turn but leaves the agent, tmux session, and compute running.

Install the fwd skill

The package includes an Agent Skills-compatible workflow. The first interactive `fwd` invocation offers to install it for Codex and Claude through `npx skills`. You can also install it directly:

```
npx skills add Sid-MB/fwd --skill fwd -g -a codex -a claude-code
```

Invoke the installed skill as `$fwd …` in Codex or `/fwd …` in Claude Code. The skill uses non-attaching, machine-readable commands and returns an exact `fwd attach SESSION` instruction when a human terminal is needed.

Runtime policy

Remote VMs and allocations are treated as the isolation boundary, so registered agents default to full access without an additional agent sandbox. Change that for less-trusted targets:

```
[agents.codex]
full_access = false
args = ["--sandbox", "workspace-write", "--ask-for-approval", "on-
     request"]
environment = {}
```

Explicit permission arguments take precedence. Environment entries are defaults and do not replace values already exported by the remote shell.

Troubleshooting

Start with diagnostics

```
fwd doctor
fwd doctor --json
fwd ls --all-projects --json
```

`fwd doctor` checks local prerequisites and configured target reachability. Structured output keeps data on stdout and diagnostics on stderr.

Recover a partial launch

`fwd up` is idempotent. Rerun the same launch after an interrupted setup, dependency failure, or transient provider error.

If the target is running and synchronized but launch preparation cannot create the primary tmux session, open a plain recovery shell from a human terminal:

```
fwd attach SESSION --raw
```

Repair the remote environment, exit the recovery shell, and rerun the normal `fwd up` command. `--raw` does not authorize restarting stopped billable compute.

Stopped compute

Non-interactive commands never restart stopped billable compute implicitly. After confirming the cost and target, authorize it explicitly:

```
fwd attach SESSION --restart
```

Dirty remote worktree

`fwd stop`, `fwd rm`, and stop-after refuse to shut down a reachable session with uncommitted or untracked remote Git changes. Inspect and retrieve work before retrying:

```
fwd diff SESSION
fwd pull --name SESSION
```

Force flags bypass this protection and may destroy remote-only changes. Use them only after accepting that loss.

Synchronization limits

Uploads stop when compressed outbound data exceeds `sync.max_size_gb` (1 GB by default). The incomplete staging directory is discarded and the live remote project remains unchanged. Exclude accidental datasets or build outputs in `.fwdignore`; raise the project limit only for a deliberately large project:

```
fwd config set --project sync.max_size_gb 4
```

Destructive operations

`fwd rm` permanently deletes remote resources. `fwd uninstall` removes only local fwd data and can leave remote resources running and billing. Remove tracked sessions first:

```
fwd rm --all
fwd uninstall
```

If a command still fails, include `fwd info`, `fwd doctor`, the exact command, and its stderr when opening an issue.

© 2026 Siddharth M. Bhatia. All rights reserved.